

Introduction to Computer Programming

Prof. Alessio Martino

`amartino@luiss.it`

Department of Business and Management
Viale Romania 32, 00197 Rome



A.Y. 2024-2025

Outline

- 1 Exercise 1
- 2 Exercise 2
- 3 Exercise 3
- 4 Exercise 4
- 5 Exercise 5
- 6 Exercise 6
- 7 Exercise 7
- 8 Exercise 8
- 9 References

Exercise 1: Coin flipping

- Ask the user how many coin tosses have to be performed
- Keep track of the number of times we get heads and tails
- We assume a *fair coin* (i.e., unbiased 50% chance of getting head and tail at each toss)

Hint

You may use the `random.randint()` function to simulate a single coin toss.

```
import random

tail = 0
head = 0

n = int(input("How many coin tosses? "))
for i in range(n):
    x = random.randint(1, 100)
    if x > 50:
        head = head + 1
    else:
        tail = tail + 1

print("Number of heads: ", head)
print("Number of tails: ", tail)
```

Exercise 2: Sum of sequence

Write a function named `rolling_sum()` that:

- repeatedly asks the user to insert a number
- we assume integer numbers
- keeps track of the sum of the numbers inserted so far
- if the user inserts 0, then exit and return the sum

```
def rolling_sum():  
    this_number = int(input("Insert a number: "))  
    my_sum = this_number  
  
    while this_number != 0:  
        this_number = int(input("Insert a number: "))  
        my_sum = my_sum + this_number  
  
    return my_sum  
  
print("Total sum is: ", rolling_sum())
```

Exercise 3: Odd or even?

Write a function named `odd_or_even()` that:

- accepts an integer number, say n
- prints the number of odd numbers in range $[1, n]$ (i.e., both included)
- prints the number of even numbers in range $[1, n]$ (i.e., both included)

```
def odd_or_even(n):  
    odd = 0  
    even = 0  
  
    for i in range(1, n + 1):  
        if i % 2 == 0:  
            even += 1  
        else:  
            odd += 1  
  
    print("Number of odd elements: ", odd)  
    print("Number of even elements: ", even)
```

Exercise 4: The Fibonacci Sequence

Write a function named `fibonacci1()` that:

- accepts an integer number, say n
- prints the first n elements of the Fibonacci sequence.

Definition

Each number in the Fibonacci sequence is given by the sum of the two preceding ones.

The Fibonacci sequence starts with two 'seed' numbers (0 and 1) and then the proper sequence starts:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

```
def fibonacci1(n):  
    second_to_last = 0  
    last = 1  
  
    print(second_to_last)  
    print(last)  
  
    for i in range(n + 1):  
        new = last + second_to_last  
        print(new)  
        second_to_last = last  
        last = new
```

Exercise 5: Multiples

Write a function named `multiples()` that:

- accepts an integer number, say n
- iterates the integers in $[1, n]$
- for multiples of 3, print 'Mult3' instead of the number
- for multiples of 5, print 'Mult5' instead of the number
- for multiples of 5 and 3, print 'Mult35' instead of the number

```
def multiples():  
    for num in range(1, n + 1):  
        if num % 3 == 0 and num % 5 == 0:  
            print("Mult35")  
        elif num % 3 == 0:  
            print("Mult3")  
        elif num % 5 == 0:  
            print("Mult5")  
        else:  
            print(num)
```

Exercise 6: Print a subset

Write a function named `print_subset()` that:

- accepts an integer number, say n
- accepts an integer number, say $m \leq n$
- scans all integer numbers in range $[1, n]$, but prints all numbers in range $[1, m]$

Solution #1:

```
def print_subset(n, m):
    for num in range(1, n + 1):
        if num <= m:
            print(num)
```

Solution #2:

```
def print_subset(n, m):
    for num in range(1, n + 1):
        print(num)
        if num == m:
            break
```

Stop and think

Under some circumstances, the second solution is more efficient than the first. What are those circumstances and why is it more efficient?

Exercise 7: Primality test

Write a function named `smallerPrimes()` that:

- accepts a positive integer number, say n
- prints all positive integer prime numbers smaller than n

Recall

A *prime* number is a natural number greater than 1 that has no positive divisors other than 1 and itself.

Hint

The simplest primality test aims at checking whether n is evenly divisible (i.e., division with no remainder) by any number in range $[2, n/2]$. If so, the number is not prime.

```
def is_prime(num):  
    if num <= 1:  
        return False  
    for i in range(2, int(num/2) + 1):  
        if num % i == 0:  
            return False  
    return True  
  
def smallerPrimes(n):  
    for i in range(2, n):  
        if is_prime(i):  
            print(i)
```

Exercise 8: Pythagorean triples

Write a function named `pythagoreanTriples()` that:

- accepts an integer positive number, say n
- prints all x and y pairs such that $x^2 + y^2 = n^2$

Recall

The Pythagorean triple consists of three positive integers x , y and n , with $x, y \leq n$, such that:

$$x^2 + y^2 = n^2$$

```
def pythagoreanTriples(n):  
    for x in range(1, n):  
        for y in range(1, n):  
            if x**2 + y**2 == n**2:  
                print(x, y)
```

References

- Guttag, J. V. (2013). *Introduction to Computation and Programming using Python (Revised and Expanded Edition)*. MIT Press. [Sections 2.4, 3.2]
- Downey, A. (2015). *Think Python*. Green Tea Press ed. 2nd. [Sections 4.2, 7.3, 7.4]